



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Reti Logiche

Lezioni, e Simulazioni d'esame

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

ottobre 2026

Indice

1	Introduzione al corso di Reti Logiche	4
1.1	Informazioni sul corso	4
1.1.1	Valutazione	4
1.1.2	Materiale e testi	4
1.1.3	Pre-requisiti	4
1.2	Struttura del corso	5
1.3	Perché digitale?	5
1.3.1	Segnale analogico	5
1.3.2	Segnale digitale	5
1.3.3	Più segnali per una grandezza continua	5
1.3.4	Vantaggi e svantaggi dei segnali digitali	6
1.4	Tecnologie di implementazione	6
1.5	Applicazioni	6
1.6	Obiettivi del corso	7
2	Funzioni booleane, algebra di Boole e semplificazione logica	7
2.1	Funzioni booleane	7
2.1.1	Notazione	7
2.1.2	Operatori fondamentali	7
2.1.3	Tabella della verità	8
2.1.4	Dominio e dimensione delle tabelle	8
2.1.5	Espressioni	9
2.1.6	Uso delle funzioni logiche: esempi di progetto	10
2.2	Algebra di Boole	11
2.2.1	Manipolazioni algebriche	11
2.2.2	Assiomi	12
2.2.3	Teoremi	12
2.3	Forme canoniche	12
2.3.1	Dalla tabella della verità a un'espressione	12
2.3.2	Teorema di espansione di Shannon	13
2.3.3	Espansione completa (prima forma canonica)	13
2.3.4	Seconda forma canonica	14
2.3.5	SOP e POS	15
2.3.6	Terminologia	15
2.4	Sintesi a due livelli	15
2.5	Implicanti	16
2.6	Mappe di Karnaugh	17
2.6.1	Interpretazione geometrica	17
2.6.2	La mappa di Karnaugh	18
2.6.3	Adiacenze a tre variabili	18
2.7	Scelta degli implicanti	19
2.7.1	Implicanti primi, ridondanti, essenziali	19
2.7.2	Procedimento	19
2.7.3	Esempio: $f = b + ac$	20
2.8	Esempi con funzioni a 3 variabili	20
2.8.1	Scacchiera (odd function)	20
2.8.2	Conta uni	20

2.8.3	Priority encoder	21
2.9	Mappe a 4 e più variabili	23
2.9.1	Esempio 1	23
2.9.2	Esempio 2	24
2.10	Uso delle porte logiche e insiemi completi	24
2.10.1	Porte con ingressi costanti	24
2.10.2	Insiemi di operatori completi	24
3	Circuiti combinatori fondamentali	25
3.1	Metodo gerarchico	25
3.2	Multiplexer	25
3.2.1	Multiplexer 4 a 1	25
3.2.2	Simbolo grafico e usi	25
3.3	Decodifiche (decoder)	26
3.3.1	Decodifica 1 a 2	26
3.3.2	Decodifica 2 a 4	26
3.3.3	A cosa servono le decodifiche	27
3.4	Demultiplexer	27
3.4.1	Realizzazione con decodifica	27
3.5	Encoder o codifica	27
3.5.1	Priority encoder	28
3.6	Decodifica per display a 7 segmenti	29
3.6.1	Minimizzazione congiunta	30
3.7	Take away: blocchi base	30
3.8	Indifferenze o don't care	31
3.8.1	Le indifferenze	31
3.8.2	Dimensione degli implicanti	31
3.8.3	Come si usano le indifferenze	32
3.8.4	Take away: indifferenze	32
3.9	Mappe a 5 e più variabili	32
3.9.1	Mappe a 5 variabili	32
3.9.2	Mappe a 6 e più variabili	34
3.10	Altri modi di sintetizzare: NAND–NAND e NOR–NOR	34
3.10.1	Realizzazione NAND–NAND	34
3.10.2	Realizzazione NOR–NOR	34
3.10.3	Take away: alternative realizzative	35
4	Tempistiche e diagrammi temporali	35
4.1	Perché le tempistiche contano	35
4.2	Diagramma temporale	35
4.2.1	Diagramma dell'invertitore	36
4.2.2	Rete logica: il multiplexer	36
4.2.3	Come completare un diagramma	37
4.2.4	Rete logica: il multiplexer con ritardi	37
4.2.5	Osservazioni	38
4.3	Alee (glitch)	38
4.3.1	Alee nel multiplexer	38
4.3.2	Alee sulla mappa di Karnaugh	39
4.3.3	Come prevenire le alee	39

4.4	Take away	40
-----	---------------------	----

1 Introduzione al corso di Reti Logiche

1.1 Informazioni sul corso

- **Corso:** *Fondamenti di Elettronica Digitale – Reti Logiche*, A.A. 2026/2027.
- **Collocazione:** secondo anno, laurea triennale in **Ingegneria Informatica, delle Comunicazioni ed Elettronica** e in **Informatica**; **48 ore, 6 crediti**.
- **Docenti:**
 - Prof. **Roberto Passerone** – tel. 0461 28-3971, roberto.passerone@unitn.it; ufficio al secondo piano, lato valle, Povo 2; ricevimento su appuntamento.
 - Ing. **Gianpietro Maddinelli** – gmaddinelli@fbk.eu.
- **Orario:** martedì 8:30–10:30 (aula A101); giovedì 13:30–15:30 (aula B109).

1.1.1 Valutazione

- **Esame scritto (obbligatorio):** domande di teoria e **progetto di una macchina a stati**; valutato tra 0 e 30.
- **Esame orale** (facoltativo): solo se allo scritto si è ottenuto **almeno 15**; aumenta o diminuisce il voto dello scritto nell'intervallo $[-3, +3]$.
- **Provette:** non previste.

1.1.2 Materiale e testi

- **Slide e dispense:** disponibili sul sito del corso su ESSE3.
- **Circuiti combinatori e sequenziali**
 - Mano–Kime, *Reti logiche*, quinta edizione, Pearson, Addison Wesley.
 - Mano–Kime, *Logic and Computer Design Fundamentals*, Pearson, Prentice Hall.
- **VHDL, FPGA**
 - Volnei Pedroni, *Circuit Design with VHDL*, third edition, The MIT Press.
 - Mark Zwolinski, *VHDL – Progetto di sistemi digitali*, Pearson, Prentice Hall.
 - Clive Maxfield, *The Design Warrior's Guide to FPGAs*, Elsevier.

1.1.3 Pre-requisiti

- **Logica matematica:** operatori logici, dimostrazioni, ecc.
- Un **linguaggio di programmazione classico** (tipo C, C++, Java).
- Familiarità con i **sistemi di sviluppo** (compilatori, sistemi integrati IDE, ecc.).

1.2 Struttura del corso

Il corso procede in tre fasi:

1. **Circuiti combinatori** – *mettere assieme porte logiche per fare cose utili*
 - multiplexer, decodifiche, encoder, aritmetica, semplificazione logica.
2. **Circuiti sequenziali** – *sequenzializzare le operazioni, gestire la memoria*
 - macchine a stati, sintesi, metodo RTL.
3. **Laboratorio** – *descrivere il sistema, simulare, implementare*
 - linguaggi ad alto livello, **VHDL**, simulazione, sintesi e implementazione, dispositivi **FPGA**.

1.3 Perché digitale?

1.3.1 Segnale analogico

- **Segnale continuo**: segnale *proporzionale* alla grandezza che si vuole rappresentare.
 - semplice ed intuitivo;
 - un solo segnale porta **tutta** l'informazione.
- **Influenzato da rumore**: il rumore **si somma** al segnale e quindi **altera la rappresentazione**; va mantenuto entro limiti accettabili.

1.3.2 Segnale digitale

- **Segnale discreto**: può assumere **solo due livelli**.
 - si definisce una **soglia** a metà tra i due livelli;
 - interpretazione: sopra la soglia $\Rightarrow$ **alto**, sotto la soglia $\Rightarrow$ **basso**;
 - **precisione limitata!**
- **Non influenzato da rumore**, se il rumore non provoca il sorpasso della soglia.
 - la **distanza dalla soglia** determina il **marginale di rumore** sopportabile.

1.3.3 Più segnali per una grandezza continua

- **Un solo segnale è insufficiente**: permette di discriminare solo due livelli (0 e 1) $\Rightarrow$ **quantizzazione troppo cruda**.
- Si usano allora **più segnali** per grandezza:
 - due segnali permettono di discriminare 4 livelli: 00, 01, 10, 11;
 - in generale n **segnali discriminano 2^n livelli**;
 - la grandezza è rappresentata come **numero binario**.

- **Precisione controllata** scegliendo il numero di segnali: in generale **non dipende dalle tolleranze dei componenti**.

Relazione tra livelli e segnali

Se con n segnali posso rappresentare $v = 2^n$ livelli, allora per rappresentare v livelli ho bisogno di

$$n = \log_2(v) \quad \text{segnali.}$$

Il numero di segnali **non cresce così in fretta**: dal grafico delle slide, ad esempio, 65536 livelli richiedono solo 16 segnali.

1.3.4 Vantaggi e svantaggi dei segnali digitali

- **Vantaggi**
 - il rumore, se contenuto, **non produce variazioni** sul segnale interpretato come cifra binaria;
 - il **numero di cifre** stabilisce la **precisione** (controllabile);
 - le **tolleranze**, se contenute, non influiscono sul funzionamento;
 - facilità di **memorizzazione** ed **integrazione**.
- **Svantaggi**
 - necessari **più segnali** per rappresentare grandezze continue;
 - apparentemente i circuiti sono **più grossi**;
 - i **trasduttori** operano solitamente con segnali continui $\Rightarrow$ servono trasformazioni dei segnali analogici in digitali, e viceversa (**ADC** e **DAC**).

1.4 Tecnologie di implementazione

Tecnologia	Prestazioni	Costi
Full custom	massime	alti
Standard Cell / ASIC	alte	alti
Programmable Logic / FPGA	medio/alte	più contenuti
Embedded software	basse	bassi

1.5 Applicazioni

- **Microprocessori e microcontrollori**: il *work-horse* del digitale.
- **GPU**: parallelismo, high throughput.
- **AI**: per esempio guida autonoma, analisi dei dati.
- **Server farm**: database, analisi finanziaria, genomica.
- **Signal processing**: medical imaging, filtraggio, SDR, MIMO.
- **Security**: crittografia, currency mining.

1.6 Obiettivi del corso

1. **Comprendere** il funzionamento dei circuiti e delle reti logiche combinatorie e sequenziali.
2. Saper **gestire la complessità** del progetto tramite **decomposizione**.
3. **Realizzare** sistemi digitali in **VHDL** e verificarne il funzionamento tramite **simulazione** ed **implementazione su schede FPGA**.

2 Funzioni booleane, algebra di Boole e semplificazione logica

2.1 Funzioni booleane

2.1.1 Notazione

- **Dominio**: l'insieme contiene solo due elementi, $B = \{0, 1\}$.
- **Variabili**: si indicano con le lettere x, y, z oppure $a, b, c, \dots$
- **Funzione scalare** di n variabili: $f(x_1, \dots, x_n) : B^n \rightarrow B$
 - ogni variabile prende valore 0/1 e il risultato vale solo 0/1.
- **Funzione vettoriale**: $F(x_1, \dots, x_n) : B^n \rightarrow B^m$
 - si tratta come m **funzioni indipendenti** di n variabili ciascuna.

2.1.2 Operatori fondamentali

Rappresentano i **connettivi logici** fondamentali.

- **AND**(x, y) : $B^2 \rightarrow B$, indicato come $x \cdot y$ oppure xy
 - vale 1 se e solo se **entrambe** le variabili valgono 1;
 - chiamata anche *congiunzione*, *intersezione*, *greatest lower bound*.
- **OR**(x, y) : $B^2 \rightarrow B$, indicato come $x + y$
 - vale 1 se e solo se **almeno una** delle due variabili vale 1;
 - chiamata anche *disgiunzione*, *unione*, *least upper bound*.
- **NOT**(x) : $B \rightarrow B$, indicato come $\bar{x}$, x' , $\sim x$
 - vale 1 se x vale 0, e vale 0 se x vale 1;
 - chiamata anche *complementazione* o *negazione*.

2.1.3 Tabella della verità

- Una funzione è definita **enumerando i valori** che assume in corrispondenza a ciascuna combinazione di ingressi.
- Invece di un “grafico” si rappresenta la funzione tramite una tabella, detta **tabella della verità**.

Altri operatori a 2 variabili

- **NAND**: $(x \cdot y)'$ **NOR**: $(x + y)'$
- **EXOR (XOR)**: $x \oplus y = xy' + x'y$ **EXNOR**: $(x \oplus y)' = x'y' + xy$

x	y	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

x	NOT
0	1
1	0

Altri esempi

- **Confronto**: $f(x, y)$ vale 1 solo se $x = y$ (stessa tabella dello XNOR).
- **Confronto multiplo**: $f(a, b, c)$ vale 1 solo se $a = b$ e $b \neq c$.

x	y	Conf
0	0	1
0	1	0
1	0	0
1	1	1

a	b	c	$f(a, b, c)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

2.1.4 Dominio e dimensione delle tabelle

- Il numero delle possibili combinazioni dei valori delle variabili è **finito**: con n variabili e 2 soli valori sono possibili solo 2^n combinazioni.

- Esempi:

$$B^2 = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$$

$$B^3 = \{(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)\}$$

- Una tabella con n ingressi ha 2^n righe, cioè (un bit di uscita per riga) 2^{n-3} byte:

n	2^{n-3}	dimensione
4	2^1	2 byte
8	2^5	32 byte
16	2^{13}	8 kbyte
32	2^{29}	512 Mbyte
36	2^{33}	8 Gbyte

- Le tabelle sono quindi **difficilmente manipolabili**; funzioni di decine di variabili sono normali (un sommatore a 16 bit ha $32 + 1$ ingressi).
- Occorre trovare una **rappresentazione molto più compatta** delle funzioni.

2.1.5 Espressioni

- Una funzione può essere rappresentata come **espressione** che fa uso degli operatori già definiti (AND, OR, NOT).
- Per convenzione l'**AND** ha **precedenza sulla OR**.
- Esempio: $f(a, b, c) = ab + b'c$

a	b	c	$f(a, b, c)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Dall'espressione al circuito. L'espressione si realizza **combinando i circuiti delle porte logiche**, partendo dall'**operatore più esterno** (l'ultimo). Esempio: $f(a, b, c) = a'(b(a + c)' + b'(a + c'))$

1. operatore più esterno: $f = a' \cdot (\dots) \Rightarrow$ **porta AND**;
2. operatore successivo: $f = a' \cdot (\dots + \dots) \Rightarrow$ **porta OR**;

3. successivo ancora: $f = a' \cdot (b \cdot (\dots)' + b' \cdot (\dots)) \Rightarrow$ **due porte AND** e complementazioni;
4. infine: $f = a' (b(a + c)' + b'(a + c')) \Rightarrow$ **due porte OR**.

Dal circuito all'espressione. Si procede a ritroso **partendo dall'uscita**. Esempio (circuito un po' diverso dal precedente):

$$\begin{aligned} f(a, b, c) &= a' \cdot (\dots) \\ f(a, b, c) &= a' \cdot (\dots + \dots) \\ f(a, b, c) &= a' \cdot (b \cdot (\dots) + (\dots) \cdot b') \\ f(a, b, c) &= a' (b(a + c)' + (a + c')b') \end{aligned}$$

Più espressioni per la stessa funzione.

- Diverse espressioni possono rappresentare la **medesima funzione**: $f(a, b, c) = ab + b'c = a'b'c + ab'c + abc' + abc$ (verificare che abbiano la stessa tabella).
- La **dimensione** dell'espressione è meno influenzata dal numero di variabili: dipende da **cosa la funzione calcola**, ed è difficile da determinare a priori.

2.1.6 Uso delle funzioni logiche: esempi di progetto

Esercizio: tende da sole

Abbassare le tende **se c'è il sole o se si preme un comando**, a meno che non ci sia **troppo vento**.

Variabili: **sole** (0 non c'è, 1 c'è), **comando** (0 non attivo, 1 attivo), **vento** (0 non c'è vento, 1 c'è troppo vento), **tende** (0 alzate, 1 abbassate).

Soluzione: le tende si abbassano (**tende** = 1) se e solo se **non c'è vento**, e c'è il sole oppure (inclusivo) è stato dato il comando.

$$\text{tende} = \text{vento}' \cdot (\text{sole} + \text{comando})$$

sole	comando	vento	tende
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Esercizio: spia della cintura di sicurezza

Realizzare un dispositivo per automobili che accenda una spia qualora l'automobile sia accesa e la cintura di sicurezza non sia allacciata.

Derivare la funzione logica che rappresenta la specifica e, da questa, disegnare il circuito corrispondente.

Soluzione: variabili **spia** (uscita: 0 spenta, 1 accesa), **quadro** (ingresso: 0 spento, 1 acceso), **cintura** (ingresso: 0 slacciata, 1 allacciata).

$$\text{spia} = \text{quadro} \cdot \text{cintura}'$$

Circuito: una porta AND con ingressi **quadro** e **cintura** negata (tramite un invertitore).

quadro	cintura	spia
0	0	0
0	1	0
1	0	1
1	1	0

Take away.

- Ogni sistema segue una **logica di funzionamento**, esprimibile tramite **funzioni booleane**.
- Le funzioni si rappresentano con **espressioni**, dalle quali è facile ricavare un circuito.
- **Obiettivo:** manipolare funzioni complesse e gestire il funzionamento di una rete nel tempo.

2.2 Algebra di Boole

2.2.1 Manipolazioni algebriche

- Le espressioni si manipolano grazie alle **proprietà** delle operazioni fondamentali.
- Ogni proprietà esprime una **equivalenza**: due espressioni sono equivalenti se rappresentano la **medesima funzione**.
- Le proprietà servono a **derivare una nuova espressione equivalente** a quella data.
- **Come si dimostra** una proprietà, ad esempio l'idempotenza ($x + x = x$, $x \cdot x = x$)?
 - basta provare **tutte le combinazioni di ingresso** (induzione completa);
 - cioè si **confronta la tabella della verità** dell'espressione a sinistra con quella dell'espressione a destra: se sono uguali, le espressioni sono equivalenti.

2.2.2 Assiomi

Proprietà	Forma 1	Forma 2 (duale)
Identità	$x + 0 = x$	$y \cdot 1 = y$
Commutativa	$x + y = y + x$	$x \cdot y = y \cdot x$
Distributiva	$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$	$x + (y \cdot z) = (x + y) \cdot (x + z)$
Complementazione	$x + x' = 1$	$x \cdot x' = 0$

2.2.3 Teoremi

Proprietà	Forma 1	Forma 2 (duale)
Associativa	$x + (y + z) = (x + y) + z$	$x(yz) = (xy)z$
Elemento nullo	$x + 1 = 1$	$x \cdot 0 = 0$
Semplificazione	$x + x'y = x + y$	$x(x' + y) = xy$
Involuzione	$(x')' = x$	
Idempotenza	$x + x = x$	$x \cdot x = x$
Adiacenza	$xy + xy' = x$	$(x + y)(x + y') = x$
Assorbimento	$x + xy = x$	$x(x + y) = x$

- La **proprietà associativa** permette di definire in modo univoco AND e OR di più di 2 variabili (analogamente: **porte logiche a più ingressi**).
- **Esempio di dimostrazione** (assorbimento): $x + xy = x \cdot 1 + xy = x(1 + y) = x \cdot 1 = x$.

Legge di De Morgan

$$\begin{aligned} (x + y)' &= x' \cdot y' & (x' + y')' &= x \cdot y \\ (x \cdot y)' &= x' + y' & (x' \cdot y')' &= x + y \end{aligned}$$

Trasforma una **OR** in una **AND**, e viceversa (NOR e NAND).

2.3 Forme canoniche

2.3.1 Dalla tabella della verità a un'espressione

Le proprietà permettono di ridurre la dimensione delle espressioni (e quindi dei circuiti). Ma come si trova una **prima espressione** se si ha solo una tabella della verità?

- **Semplifico il problema:** si **spezza** la tabella in due, fissando $a = 0$ e $a = 1$.
- Si ottengono due funzioni di meno variabili: $f(0, b, c) = b' + c'$ e $f(1, b, c) = c$.
- **Aggiungo la condizione:** ogni pezzo vale solo se a ha il valore corrispondente, quindi $a'(b' + c')$ e ac .
- Ricombinandole: $f(a, b, c) = ac + a'(b' + c')$.

a	b	c	$f(a, b, c)$
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

2.3.2 Teorema di espansione di Shannon

Teorema di espansione di Shannon

Sia f una funzione di n variabili. Allora

$$f(x_1, \dots, x_n) = x_1 \cdot f(1, x_2, \dots, x_n) + x_1' \cdot f(0, x_2, \dots, x_n)$$

- La variabile x_1 **seleziona** una delle due forme della funzione: quella in cui si sostituisce 1 quando $x_1 = 1$, quella in cui si sostituisce 0 quando $x_1 = 0$.
- **Riduzione delle variabili:** $f(1, x_2, \dots, x_n)$ e $f(0, x_2, \dots, x_n)$ non dipendono più da x_1 .

Esempio. Sia $f(a, b, c) = a'(b' + c') + c(a + b')$. Allora

$$\begin{aligned} f(a, b, c) &= a \cdot f(1, b, c) + a' \cdot f(0, b, c) \\ &= a \cdot [1'(b' + c') + c(1 + b')] + a' \cdot [0'(b' + c') + c(0 + b')] \\ &= a \cdot [c] + a' \cdot [(b' + c') + cb'] = ac + a'(b' + c' + cb') \end{aligned}$$

con $f(1, b, c) = c$ e $f(0, b, c) = b' + c' + cb'$.

Applicazione ricorsiva. Le funzioni $f(1, x_2, \dots, x_n)$ e $f(0, x_2, \dots, x_n)$ si possono espandere a loro volta su x_2 :

$$\begin{aligned} f(x_1, \dots, x_n) &= x_1 x_2 \cdot f(1, 1, x_3, \dots, x_n) + x_1 x_2' \cdot f(1, 0, x_3, \dots, x_n) \\ &\quad + x_1' x_2 \cdot f(0, 1, x_3, \dots, x_n) + x_1' x_2' \cdot f(0, 0, x_3, \dots, x_n) \end{aligned}$$

Struttura: un prodotto di variabili moltiplica la funzione valutata sulle variabili che assumono valore **1 se in forma affermata** e **0 se in forma negata**.

2.3.3 Espansione completa (prima forma canonica)

- Espandendo su **tutte** le variabili, f valutata in un punto è un **valore preciso** (0 o 1), ottenibile dalla tabella della verità.
- Il termine prodotto è presente **se e solo se** la funzione in quel punto vale 1.

- Si scrive quindi direttamente dalla tabella, **guardando solo le righe in cui f vale 1**:
 - per ogni 1 si guarda il valore delle variabili in quella riga;
 - se alla variabile è assegnato 1, compare in forma **affermata** nel prodotto;
 - se è assegnato 0, compare in forma **negata**.

Esempio. $f(a, b, c) = a'(b' + c') + c(a + b')$ (tabella precedente). Si eliminano i termini con $f = 0$ ($a'bc$, $ab'c'$, abc'):

$$f(a, b, c) = a'b'c' + a'b'c + a'bc' + ab'c + abc$$

Per casa: disegnare il circuito corrispondente.

Forme canoniche e conseguenze.

- L'espansione completa è un'espressione **univoca** (i termini derivano dalla tabella): due funzioni uguali hanno la **stessa espansione completa**, detta **forma canonica**.
- In forma canonica due espressioni sono uguali **se e solo se** le funzioni sono uguali (non vale per le espressioni normali, che possono essere diverse a parità di funzione).
- **Qualunque funzione booleana** può essere espressa tramite gli operatori di base (basta scriverla in forma canonica): risultato non ovvio, non vale per esempio in analisi.
- Bastano in realtà **due soli operatori** (De Morgan): $x + y = (x' \cdot y')'$ (OR con sole AND e NOT) e $x \cdot y = (x' + y')'$ (AND con sole OR e NOT). Uno degli operatori di base è **superfluo**.

2.3.4 Seconda forma canonica

Il teorema di Shannon vale anche in **forma duale**:

$$f(x_1, \dots, x_n) = (x_1 + f(0, x_2, \dots, x_n)) \cdot (x'_1 + f(1, x_2, \dots, x_n))$$

Espandendo su tutte le variabili si ottiene un prodotto di termini del tipo $(x_1 + x_2 + \dots + x_n + f(0, 0, \dots, 0))$, $\dots$, $(x'_1 + x'_2 + \dots + x'_n + f(1, 1, \dots, 1))$.

- **Dualità**: in ogni termine una variabile compare **affermata se valutata in 0, negata se valutata in 1**.
- Il termine viene **eliminato se la funzione vale 1**: partendo dalla tabella si scelgono le righe in cui la funzione **vale 0**.

Esempio. $f(a, b, c) = a'(b' + c') + c(a + b')$: le righe con $f = 0$ sono 011, 100, 110, quindi

$$f(a, b, c) = (a + b' + c') \cdot (a' + b + c) \cdot (a' + b' + c)$$

Per casa: disegnare il circuito corrispondente.

2.3.5 SOP e POS

- **Prima forma canonica = somma di prodotti** (*Sum Of Products*, **SOP**): una grossa somma in cui ogni termine è un prodotto di variabili affermate o negate.

$$- f(a, b, c) = a'b'c' + a'b'c + a'bc' + ab'c + abc$$

- **Seconda forma canonica = prodotto di somme** (*Product Of Sums*, **POS**): un grosso prodotto in cui ogni termine è una somma di variabili affermate o negate.

$$- f(a, b, c) = (a + b' + c') \cdot (a' + b + c) \cdot (a' + b' + c)$$

2.3.6 Terminologia

- **Letterale**: una variabile in forma affermata o negata (es. x, x', y', b, a').
 - $f(a, b, c) = a'b'c' + a'b'c + a'bc' + ab'c + abc$ ha 3 variabili e **15 letterali**;
 - $f(a, b, c) = a'b' + a'c' + ac$ ha 3 variabili e **6 letterali**.
- La **complessità** di un'espressione si misura dal **numero di letterali**: ogni letterale diventa l'ingresso di una porta logica, quindi il numero di letterali è correlato con il **numero di transistori** del circuito.
- **Prodotto fondamentale (minterm)**: prodotto in cui **ogni variabile compare una volta** come letterale, es. $a'b'c', a'b'c, a'bc', ab'c, abc$.
- **Somma fondamentale (maxterm)**: somma in cui ogni variabile compare una volta come letterale, es. $(a + b' + c), (a' + b + c'), (a' + b' + c), (a + b + c)$.
- Le forme canoniche usano minterm e maxterm: i **minterm** di una funzione corrispondono agli **1** della tabella della verità, i **maxterm** agli **0**.

2.4 Sintesi a due livelli

- **Livello**: massimo numero di porte logiche attraversate dall'ingresso all'uscita.
 - $f = ab + b'c$ ha **2 livelli**; $f = a'(b(a + c') + (a + c')b')$ ha **4 livelli**.
- A mano ci si limita a realizzazioni a **due livelli**:
 - teoricamente semplici da ottenere;
 - in prima analisi sono anche il circuito **più veloce**;
 - talvolta però sono di grosse dimensioni (es. addizionatore);
 - sono alla base dei metodi per realizzazioni multi-livello.
- Un'espressione a due livelli è
 - una **somma di prodotti**: primo livello tutto di AND, seguito da una OR;
 - oppure un **prodotto di somme**: primo livello tutto di OR, seguito da una AND (per la proprietà associativa).

2.5 Implicanti

Implicante

Siano f e g funzioni di n variabili. g è un **implicante** di f se e solo se, per qualunque assegnamento $(x_1, \dots, x_n)$ alle variabili,

$$\text{se } g(x_1, \dots, x_n) = 1 \text{ allora } f(x_1, \dots, x_n) = 1.$$

- Se g vale 1 in qualche punto, allora anche f vale 1; se g vale 0, f può essere **indifferentemente 0 o 1**.
- **Esempio:** sia $f(x, y, z) = xy + z$.
 - $g = xy$, $g = z$, $g = y'z$ **sono implicanti** di f ;
 - $g = x$ **non è un implicante** (per $x = 1, y = 0, z = 0$ vale $g = 1$ ma $f = 0$).

x	y	z	$f(x, y, z)$	xy	z	$y'z$	x
0	0	0	0	0	0	0	0
0	0	1	1	0	1	1	0
0	1	0	0	0	0	0	0
0	1	1	1	0	1	0	0
1	0	0	0	0	0	0	1
1	0	1	1	0	1	1	1
1	1	0	1	1	0	0	1
1	1	1	1	1	1	0	1

- Ogni implicante **copre** una parte della funzione: si può “fare a pezzi” $f(x, y, z) = xy + z = y'z + z + xy$.

Implicanti come mattoni.

- Un implicante rappresenta un “pezzo” di funzione: può coprire parte (non necessariamente tutti) degli 1 di una funzione.
- Vari implicanti rappresentano differenti pezzi, che possono anche **sovrapporsi**.
- La funzione è data dall'**OR di appropriati implicanti**.

Come trovare gli implicanti.

- Si parte dai **minterm**: sono implicanti, ma ognuno copre un solo 1 della funzione.
- Si cerca di **espanderli** con le proprietà dell'algebra booleana, mettendo assieme minterm per formare **termini più grossi** (con meno letterali).
- Ci si limita a implicanti in forma di **prodotto di letterali**: ab' va bene, $ab' + a'b$ **non va bene**.

Esercizio: multiplexer 2 a 1

Progettare un circuito che permetta di scegliere uno tra due segnali a e b tramite un comando s : quando $s = 0$ si deve scegliere a , quando $s = 1$ si deve scegliere b .

Soluzione: si scrive la tabella della verità e la **prima forma canonica**:

$$m = s'ab' + s'ab + sa'b + sab$$

s	a	b	$m(a, b, s)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Applicando l'**adiacenza logica** $xy + xy' = x$ (con x gruppo di letterali e y una variabile; sulla tabella corrisponde a **righe in cui $m = 1$ che differiscono per una sola variabile**):

$$m = s'ab' + s'ab + sa'b + sab \Rightarrow m = s'a + sb$$

Ridotto da 12 a 4 letterali. Circuito: due AND ($s'a$ e sb , con s negato da un invertitore) in ingresso a una OR; equivale al simbolo del multiplexer con ingresso b in posizione 1, a in posizione 0 e selezione s .

2.6 Mappe di Karnaugh

2.6.1 Interpretazione geometrica

- Nella mappa si chiama **term** un gruppo di 1 di f che non sia un minterm: con un solo colpo si prendono diversi 1.
- È come scrivere la forma canonica, ma si **tralasciano le variabili il cui valore cambia**: $s'ab' + s'ab$ diventa $s'a$. Si possono scrivere **direttamente**!
- Sono facili da individuare quando le righe sono una vicina all'altra, meno quando sono lontane.
- **Trucco**: disporre le righe in modo che **cambi una sola variabile alla volta**, così termini logicamente adiacenti sono vicini anche fisicamente sulla tabella.
- Invece del codice binario normale si usa il **codice Gray**: tra una riga e la successiva cambia solamente una variabile.
- **Non funziona sempre**: si ha una sola dimensione, un termine risulta ancora spezzato $\Rightarrow$ occorre sfruttare **dimensioni aggiuntive**.

s	a	b	$f(a, b, s)$
0	0	0	0
0	0	1	0
0	1	1	1
0	1	0	1
1	1	0	0
1	1	1	1
1	0	1	1
1	0	0	0

2.6.2 La mappa di Karnaugh

- Le variabili sono distribuite su **righe e colonne**, entrambe numerate in **codice Gray** (00, 01, 11, 10).
- La prima colonna è vicina all'ultima!**
- Nell'esempio del multiplexer (colonne sa , riga b): $s'a$, ab e sb .
- Il raggruppamento dei tre 1 della riga $b = 1$ **non è esprimibile con un solo prodotto**: è la **somma** dei due implicanti $ab + sb$.

$b \backslash sa$	00	01	11	10
0	0	1	0	0
1	0	1	1	1

rosso: $s'a$ **blu:** ab **verde:** sb **arancio:** casella coperta da più implicanti

Take away.

- Ci si limita a espressioni **a due livelli**; gli implicanti sono **termini prodotto**, trovabili per **adiacenza logica**.
- Si trasforma l'adiacenza logica in **adiacenza geometrica**: occorre mettere vicine nello spazio le righe della tabella che sono logicamente vicine.
- Si **raggruppano gli 1** che stanno vicini, cercando i **raggruppamenti più grossi** (hanno meno letterali).

2.6.3 Adiacenze a tre variabili

Mappa $a \backslash bc$ (colonne bc in ordine 00, 01, 11, 10), **la prima colonna è vicina all'ultima**.

- Due uni** (termini a **2 letterali**), 12 gruppi possibili:
 - riga $a = 0$: $a'b'$ (colonne 00, 01), $a'c$ (01, 11), $a'b$ (11, 10), $a'c'$ (10, 00: prima e ultima colonna);

- riga $a = 1$: ab' , ac , ab , ac' (stesse colonne);
- in verticale (stessa colonna, due righe): $b'c'$ (00), $b'c$ (01), bc (11), bc' (10).
- **Quattro uni** (termini a **1 letterale**): b' (colonne 00, 01), c (01, 11), b (11, 10), c' (10, 00), a' (riga $a = 0$), a (riga $a = 1$).
- **Tutti o nessuno**: tutti 1 $\Rightarrow f = 1$; tutti 0 $\Rightarrow f = 0$.

2.7 Scelta degli implicanti

2.7.1 Implicanti primi, ridondanti, essenziali

- **Primi**: un implicante è **primo** quando **non è contenuto in un altro implicante**.
 - gli implicanti contenuti in altri si possono **buttare via**;
 - nel multiplexer i minterm *non* sono primi ($sa'b \rightarrow sb$).
- **Ridondanti**: alcuni implicanti primi non servono.
 - per il multiplexer ci sono **3 implicanti primi**: $s'a$, sb , ab ;
 - uno dei tre non serve (ab) perché gli altri da soli già coprono la funzione.
- **Essenziali**: coprono un 1 **non coperto da nessun altro implicante primo**; devono comparire in **qualunque copertura minima**.
 - $s'a$ ed sb sono essenziali.

$s \backslash ab$	00	01	11	10
0	0	0	1	1
1	0	1	1	0

2.7.2 Procedimento

Obiettivo: trovare il **più piccolo insieme di implicanti primi** che ricoprano tutta la funzione (primi, perché altrimenti ci sarebbero implicanti con meno letterali; insieme più piccolo, per ridurre il numero di termini).

1. Scrivere la tabella della verità in forma di **mappa di Karnaugh** (numerazione in codice Gray).
2. Partendo dai minterm, **espandersi in tutte le direzioni** per trovare le adiacenze; quando non ci si può più espandere si è arrivati all'**implicante primo**.
3. Scrivere l'espressione degli implicanti guardando le **variabili che sono costanti** (basta guardare le cifre scritte sulle righe e colonne della mappa).
4. Scegliere un insieme di implicanti **non ridondante**:
 - prendere tutti gli implicanti primi **essenziali**;
 - coprire i rimanenti 1 scegliendo un insieme piccolo (cosa **non banale**).

2.7.3 Esempio: $f = b + ac$

- Implicanti primi: b e ac , **entrambi essenziali**: $f = b + ac$.
- **Nota bene**: un 1 è coperto più volte (corrisponde al minterm abc). Non è un problema: due porte AND saranno attive. Meglio che non usare $ab'c$.
- La porta AND a 1 ingresso si può togliere e collegare b direttamente alla porta OR.

$a \backslash bc$	00	01	11	10
0	0	0	1	1
1	0	1	1	1

2.8 Esempi con funzioni a 3 variabili

2.8.1 Scacchiera (odd function)

- I minterm **non sono espandibili**: sono tutti implicanti primi; la mappa a scacchiera **non è minimizzabile**: $f = ab'c' + a'b'c + abc + a'bc'$.
- Si tratta di una **rete di EXOR**:

$$\begin{aligned}
 f &= b'(ac' + a'c) + b(ac + a'c') \\
 &= b'(ac' + a'c) + b(ac' + a'c)' = b'(a \oplus c) + b(a \oplus c)' \\
 &= b \oplus (a \oplus c) = a \oplus b \oplus c
 \end{aligned}$$

Dimostrazione dell'identità usata, $(ac + a'c') = (ac' + a'c)'$ (De Morgan):

$$\begin{aligned}
 (ac + a'c') &= (ac + a'c')'' = ((ac)'(a'c')')' = ((a' + c')(a + c))' \\
 &= (a'a + a'c + c'a + cc')' = (a'c + c'a)' = (ac' + a'c)'
 \end{aligned}$$

$a \backslash bc$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

2.8.2 Conta uni

Esercizio: conta uni

Realizzare un circuito con 3 ingressi (a, b, c) e un'uscita che indichi **quanti ingressi sono a 1**.

Soluzione:

- Occorre innanzi tutto **codificare l'uscita**, per esempio in binario: da 0 a 3 ingressi a 1, cioè 4 combinazioni (0, 1, 2, 3), sono sufficienti **due cifre binarie**.
- La funzione ha **2 uscite** (s_1, s_0): si trattano come due funzioni indipendenti, ma i loro valori sono tra loro collegati (una colonna per ogni uscita nella tabella; per la semplificazione si trattano **separatamente**).

<i>a</i>	<i>b</i>	<i>c</i>	<i>s</i> ₁	<i>s</i> ₀
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

*s*₁

*s*₀

<i>a</i> \ <i>bc</i>	00	01	11	10
0	0	0	1	0
1	0	1	1	1

<i>a</i> \ <i>bc</i>	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$s_1 = ac + bc + ab$$

$$s_0 = a \oplus b \oplus c$$

2.8.3 Priority encoder

Esercizio: priority encoder a 3 ingressi

Come prima, ma si vuole l'indice dell'ingresso a 1 con valore più alto. Indici: $a \rightarrow 1$, $b \rightarrow 2$, $c \rightarrow 3$. Se $c = 1$ l'uscita vale 3; se $c = 0$ e $b = 1$ vale 2; se $c = 0$, $b = 0$ e $a = 1$ vale 1; se sono tutti a 0 vale 0.

Soluzione: utile per esempio per codificare le priorità degli interrupt (serve l'indice dell'interrupt attivo a priorità più alta).

<i>a</i>	<i>b</i>	<i>c</i>	<i>s</i> ₁	<i>s</i> ₀
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	1

$$s_1 = b + c$$

$$s_0 = ab' + c$$

$a \backslash bc$	00	01	11	10
0	0	1	1	1
1	0	1	1	1

$a \backslash bc$	00	01	11	10
0	0	1	1	0
1	1	1	1	0

$$s_1 = b + c$$

$$s_0 = ab' + c$$

Lieve modifica a s_0 . Scambiando le **ultime due righe** della tabella ($110 \rightarrow 11$, $111 \rightarrow 10$):

- $s_1 = b + c$ (invariata);
- per s_0 la scelta è più complessa: gli implicant $a'c$ e ac' sono **essenziali**, quindi ci vogliono per forza;
- il restante 1 si può coprire in **due modi**: $s_0 = a'c + ac' + ab'$ oppure $s_0 = a'c + ac' + b'c$ (quest'ultima è quella usata nel circuito della slide).

a	b	c	s_1	s_0
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	0

$$s_1 = b + c$$

$$s_0 \text{ (nuova)}$$

$a \backslash bc$	00	01	11	10
0	0	1	1	1
1	0	1	1	1

$a \backslash bc$	00	01	11	10
0	0	1	1	0
1	1	1	0	1

Take away.

- Semplificare una funzione di 3 variabili è un procedimento piuttosto semplice: attenzione solo alla **scelta degli implicant**; con un po' di pratica si fa molto in fretta.
- Con **più uscite** le si trattano separatamente, ma la **funzione vettoriale** deve essere progettata nel suo complesso (si potrebbe fare la **minimizzazione congiunta**).
- È facile costruire **blocchetti di uso generico**: multiplexer, conta uni, priority encoder.

2.9 Mappe a 4 e più variabili

- Le mappe di Karnaugh si possono scrivere anche a **4 e più ingressi**: diventano più grosse e trovare le adiacenze è più complesso, soprattutto **sopra le 5 variabili**.
- Il procedimento rimane lo stesso: trovare gli **implicanti primi**, tenere gli **essenziali**, scegliere tra gli altri in modo da coprire la funzione.
- A 4 variabili si aggiunge una variabile alle righe (sempre in codice Gray) e la mappa diventa **quadrata**: righe ab , colonne cd .

Uni nel gruppo	Letterali	Esempi
1 (minterm)	4	$ab'cd', a'bcd, abc'd', abcd$
2	3	$ab'd', a'cd, bc'd', abc$
4	2	$b'd', a'c, c'd', ab$
8	1	b', c, d', a

Un gruppo di 2^k uni elimina k variabili. Altri gruppi di 4 uni mostrati nelle slide: $b'c', bd, bc', ac, a'b', c'd$; gruppi di 8 uni: a', b', d, d' .

2.9.1 Esempio 1

$xy \backslash zw$	00	01	11	10
00	1	0	1	1
01	0	1	1	0
11	0	1	1	0
10	1	1	1	1

blu: $y'w'$ **verde:** yw (i due implicanti essenziali)

- Implicanti primi:** 6, tutti da quattro uni: $zw, xw, yw, y'w', xy', y'z$. Molti implicanti da 2 uni, ma **nessuno di essi è primo**; non ci sono implicanti da 8 uni.
- Essenziali:** $y'w'$ e yw .
- Ne servono poi **almeno altri due**: uno tra xy' e xw , uno tra zw e $y'z$.
- Per esempio: $f = y'w' + yw + xw + zw$, ovvero $f = (y \oplus w)' + w(x + z)$.

Da NON fare. Mettere implicanti **non primi**, ad esempio $f = y'w' + yw + xy'w + x'y'zw$: è una **soluzione più costosa!**

- $xy'w$ è contenuto in xw o in xy' ; $x'y'zw$ è contenuto in zw o in $y'z$.
- Usare quello con meno letterali!** Non importa se si coprono degli 1 più volte.

2.9.2 Esempio 2

$xy \backslash zw$	00	01	11	10
00	1	0	1	0
01	1	1	1	1
11	0	1	0	0
10	1	1	0	0

blu: $x'zw$ **verde:** $x'y$ **arancio:** coperta da entrambi

- **Implicanti primi:** 6 da due uni ($xz'w$, $yz'w$, $x'zw$, $y'z'w'$, $xy'z'$, $x'z'w'$) e uno da 4 uni ($x'y$).
- **Essenziali:** $x'zw$ e $x'y$.
- Come coprire i restanti 1? Si possono usare **3** implicanti oppure solo **2**: $f = x'zw + x'y + xz'w + y'z'w'$.
- Ci sono due insiemi non ridondanti e uno è più piccolo dell'altro: un metodo è cercare di **minimizzare le sovrapposizioni**.

2.10 Uso delle porte logiche e insiemi completi

2.10.1 Porte con ingressi costanti

Porta	Secondo ingresso	Uscita
AND	0 / 1	0 / a
OR	0 / 1	a / 1
NAND	a / 0 / 1	a' / 1 / a'
NOR	a / 0 / 1	a' / a' / 0
XOR	1 / 0	a' / a
XNOR	1 / 0	a / a'

- Nella tabella “ a ” come secondo ingresso significa che **i due ingressi sono collegati assieme**.
- Una NAND seguita da un NOT dà un **AND** (ab); una NOR seguita da un NOT dà un **OR** ($a + b$).
- Con le porte si possono quindi ottenere buffer, invertitori e costanti.

2.10.2 Insiemi di operatori completi

- L'algebra di Boole è definita dai tre operatori base **AND**, **OR** e **NOT**: con essi, per il teorema di Shannon, si rappresentano **tutte** le funzioni booleane.
- Altri insiemi di operatori possono essere usati per definire l'algebra: **{AND, NOT}**, **{OR, NOT}**, **{NAND}**, **{NOR}**, **{XOR, AND}**, **{XOR, OR}**.

- Si dicono **insiemi di operatori completi**: sono sufficienti per realizzare **tutte le funzioni logiche**.

3 Circuiti combinatori fondamentali

3.1 Metodo gerarchico

- Il metodo visto finora (scrivere la mappa e semplificarla) è **generico**, ma gestibile solo fino a un **numero limitato di variabili**.
- Con tante variabili la semplificazione diventa complessa:
 - perché ci sono **troppe caselle**;
 - soprattutto perché è **difficile perfino immaginare il valore** che deve avere la funzione: è complesso anche solo **scrivere** le mappe (e non solo perché sono di grandi dimensioni).
- Conviene costruire il circuito a partire da **blocchi base**: lì si può progettare con cura, se ne conosce bene il funzionamento e si sa già che funzionano.
- In pratica si **aumenta il livello di astrazione**: il metodo è detto **gerarchico** perché, una volta realizzato un blocco, **non lo si apre più**.

3.2 Multiplexer

3.2.1 Multiplexer 4 a 1

- Funzione con **4 ingressi di dato + 2 ingressi di controllo**: ingressi a, b, c, d , più s_0 e s_1 per la scelta $\Rightarrow$ **6 ingressi in totale**, difficile da gestire come tabella o come mappa.
- Si può fare un **circuito gerarchico**:
 1. prima si sceglie tra a e b , e tra c e d , usando s_0 (due multiplexer 2 a 1);
 2. poi si sceglie tra i due che sono rimasti con s_1 (un terzo multiplexer 2 a 1).
- Il circuito ottenuto **non è più a 2 livelli**; si potrebbe fare una soluzione a due livelli, forse **più veloce**.

Funzione del multiplexer 4 a 1

$$m(a, b, c, d, s_0, s_1) = s_0 s_1 b + s_0 s_1' d + s_0' s_1 a + s_0' s_1' c$$

3.2.2 Simbolo grafico e usi

- Nel simbolo si aggiungono gli **ingressi di dato** e quelli di **selezione**; i numerini identificano l'**ingresso attivo** in corrispondenza di una combinazione degli ingressi di selezione.
- I multiplexer sono utili in molteplici applicazioni:

- per **condividere risorse**;
- per **fare delle scelte**;
- per realizzare **dispositivi programmabili**;
- per **selezionare celle di memoria**.

3.3 Decodifiche (decoder)

- È utile poter **passare da una codifica a un'altra**: per esempio dal codice binario al codice Gray, oppure da fixed point a floating point.
- **Decodificare** significa passare da n a m cifre, con $n \leq m \leq 2^n$, in modo che a ogni codice di ingresso corrisponda **un'uscita unica** (funzione **iniettiva**).
- In pratica si **scompatta** la rappresentazione: con m cifre potresti rappresentare 2^m codici, ma ne uso solo 2^n .

3.3.1 Decodifica 1 a 2

- Un ingresso e due uscite: la prima uscita vale 1 se l'ingresso vale 0, la seconda vale 1 se l'ingresso vale 1.
- Le uscite **non sono mai a 1 o a 0 contemporaneamente**: in teoria 2 uscite potrebbero codificare 4 elementi, ma alcune combinazioni non vengono utilizzate.
- Tabella della verità (solo due righe e due uscite; **non è una mappa di Karnaugh!**):

a	x	y
0	1	0
1	0	1

- Per ispezione: $y = a$ e $x = a'$.
- **Circuito**: un invertitore per x , un collegamento diretto per y .

3.3.2 Decodifica 2 a 4

- Due ingressi e quattro uscite: la prima uscita vale 1 se l'ingresso vale 0, la seconda se vale 1, la terza se vale 2, la quarta se vale 3.
- **Ogni uscita ha un solo minterm**: inutile farsi le mappe di Karnaugh, **non c'è nulla da semplificare**.

a	b	w	x	y	z
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- Dalla tabella: $w = a'b'$, $x = a'b$, $y = ab'$, $z = ab$.
- **Circuito:** costruito con **due decodifiche 1 a 2** (una per a e una per b) che alimentano 4 porte AND.

3.3.3 A cosa servono le decodifiche

- Molto utili per **indirizzare le memorie**: il processore mette sul bus un indirizzo a n bit.
 - **metà** dei bit indirizza le **righe** con una **decodifica**;
 - l'**altra metà** sceglie la **colonna** con un **multiplexer**.
- Esempio: $n = 8$, $n/2 = 4 \Rightarrow$ decodifica 4 a 16 e multiplexer 16 a 1, per **256 celle di memoria** in una matrice 16×16 .
- Hanno molti altri usi.

3.4 Demultiplexer

- Fa il **lavoro contrario** del multiplexer: prende in ingresso una variabile binaria e la presenta su **una uscita a scelta tra 2^n possibili**.
- Le uscite non selezionate vengono tenute al valore **0**.
- Il demultiplexer ha $n + 1$ ingressi (n per la selezione, più un ingresso di dato) e 2^n uscite.
- **Esempio: demux 1 a 4:** 2 ingressi di selezione ($s_1 s_0$), 4 uscite (a, b, c, d per $s_1 s_0 = 00, 01, 10, 11$); facilmente realizzabile a due livelli con quattro mappe.

3.4.1 Realizzazione con decodifica

- Il demultiplexer si può realizzare **utilizzando una decodifica**:
 - delle porte AND **abilitano** l'ingresso x a passare a ognuna uscita;
 - la **decodifica** sceglie **quale porta AND abilitare**;
 - semplicissimo fare demux a tante uscite usando decodifiche più grosse.
- In formule (ricavate dalla struttura): $a = x \cdot s'_1 s'_0$, $b = x \cdot s'_1 s_0$, $c = x \cdot s_1 s'_0$, $d = x \cdot s_1 s_0$.

3.5 Encoder o codifica

- È il **processo contrario della decodifica**: per esempio si può passare da **8 fili a 3**.
- È molto **più semplice da realizzare**.

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

- **Ogni uscita vale 1 per una determinata combinazione di ingressi:** ad esempio A_2 vale 1 per D_4 , D_5 , D_6 e D_7 .

$$A_2 = D_4 + D_5 + D_6 + D_7$$

$$A_1 = D_2 + D_3 + D_6 + D_7$$

$$A_0 = D_1 + D_3 + D_5 + D_7$$

3.5.1 Priority encoder

- Già visto in precedenza: da 3 ingressi a 2 uscite (nessuna, 1, 2, 3: 4 combinazioni).
- Con **4 ingressi** occorrono **3 uscite**: bisogna codificare anche il caso in cui **nessun ingresso sia attivo** (nessuna, 1, 2, 3, 4: 5 combinazioni).
- Si può codificare l'uscita in modo alternativo:
 - un'uscita V (*valid*) dice se **almeno un ingresso è attivo**;
 - le altre 2 uscite (A_1 , A_0) codificano l'**indice dell'ingresso attivo con priorità più alta**;
 - si può cominciare a contare da 0 invece che da 1.

D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

- Gli ingressi a **x** indicano che il valore della variabile **non è importante** (come una *wildcard*): una x indica 2 righe contemporaneamente, due x indicano 4 righe. È solo un modo per scrivere la tabella più velocemente, altrimenti non cambia nulla.
- Per V basta mettere in **OR tutti gli ingressi**: $V = D_3 + D_2 + D_1 + D_0$. Per le altre uscite si costruiscono le mappe di Karnaugh:

$$A_1 = D_2 + D_3$$

$$A_0 = D_3 + D_1 D_2'$$

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$D_3 D_2 \backslash D_1 D_0$	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

A_1 : verde D_2 , giallo D_3 A_0 : blu D_3 , rosso $D_1 D_2'$ arancio: sovrapposizione

3.6 Decodifica per display a 7 segmenti

Esercizio: decodifica per display a 7 segmenti

Realizzare una decodifica in grado di pilotare un **display a 7 segmenti**. Si ha in ingresso un numero binario a 4 cifre (x, y, z, w); si devono calcolare **7 uscite**, ognuna in corrispondenza di un segmento. Il dato binario va rappresentato in **esadecimale** (cifre da 0 a F).

Soluzione:

- Ogni segmento è identificato da una lettera: a (alto), b (alto a destra), c (basso a destra), d (basso), e (basso a sinistra), f (alto a sinistra), g (centrale).
- Ogni segmento è **acceso** (uscita 1) oppure **spento** (uscita 0) a seconda della combinazione degli ingressi $xyzw = 0000, 0001, \dots, 1111$.
- Si semplifica **ciascun segmento separatamente**, isolandone la mappa.

segmento a

$xy \backslash zw$	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	1	0	1	1
10	1	1	0	1

segmento g

$xy \backslash zw$	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	0	1	1	1
10	1	1	1	1

Segmento a .

- Implicanti primi:** zw' , $x'z$, yz , $y'w'$, $x'yw$, $xy'z'$, xw' .
- L'unico **implicante primo non essenziale** è zw' ; quelli essenziali coprono la funzione.
- $a = x'z + yz + y'w' + xw' + x'yw + xy'z' \Rightarrow$ **14 letterali**.

Segmento g .

- **Implicanti primi:** zw' , xz , xw , $yz'w$, xy' , $x'yz'$, $y'z$, $x'yw'$.
- **Essenziali:** $y'z$ e xy' ; degli altri si possono prendere per esempio i seguenti.
- $g = y'z + xy' + zw' + xw + x'yz' \Rightarrow$ **11 letterali**.

3.6.1 Minimizzazione congiunta

- Il termine $xy'z'$ è un **implicante primo essenziale** per il segmento a .
- Lo stesso termine è implicante per il segmento g , ma **non è primo**, essendo contenuto in xy' , che è implicante primo essenziale per g .
- Il termine è **già disponibile nella rete logica**: può essere vantaggioso usarlo anche per g (gli altri 1 di xy' sono coperti nell'espressione di g da altri implicanti).

$$g = y'z + \textcolor{red}{xy'z'} + zw' + xw + x'yz'$$

- Sebbene vi siano **12 letterali**, un termine non contribuisce perché già presente nella rete logica: in totale **solì 9 letterali**.

segmento a

$xy \backslash zw$	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	1	0	1	1
10	1	1	0	1

segmento g

$xy \backslash zw$	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	0	1	1	1
10	1	1	1	1

giallo: $xy'z'$

rosso: xy'

arancio: $xy'z'$ contenuto in xy'

3.7 Take away: blocchi base

- Abbiamo realizzato un gran numero di **elementi base**: **multiplexer**, **encoder**, **decoder**, **transcoder**.
- Mettendoli assieme si possono fare **circuiti più complessi**: non necessariamente a due livelli, ma facili da capire.
- **Minimizzazione congiunta**: può fornire risultati migliori, ma è molto più complicato farla a mano $\Rightarrow$ meglio usare **programmi per calcolatore specializzati**.

3.8 Indifferenze o don't care

3.8.1 Le indifferenze

- In certe occasioni il valore dell'uscita in corrispondenza di alcune combinazioni di ingressi è **irrilevante**:
 - per esempio, nella codifica 8 a 3, quando vi sono **due o più ingressi a 1** contemporaneamente;
 - ogni volta che gli ingressi codificano un numero di combinazioni **inferiore a 2^n** .
- Le mappe di Karnaugh però devono includere un valore per **tutte** le combinazioni di ingressi: che valore dare?
 - se l'ingresso non si presenta mai, si può dare il valore 0 o 1 **indifferentemente**;
 - si assegna il valore che consente di ottenere un'espressione **più semplice**.
- Le combinazioni di ingresso per cui non si indica un valore preciso dell'uscita si chiamano **indifferenze** o **don't care** (nelle mappe: $-$).

3.8.2 Dimensione degli implicanti

- Implicanti che coprono **molti 1** (cioè più grossi) sono quelli con **meno letterali**: per realizzare un'espressione minima si prendono gli **implicanti primi**, che sono i più grossi per definizione.
- Si usano allora le indifferenze per **ingrandire gli implicanti**: si immagina che le indifferenze siano a 1, così alcuni implicanti si espandono a coprire gli 1 indifferenti.

Esercizio: display a 7 segmenti solo con cifre decimali

Realizzare la decodifica per display a 7 segmenti che mostri **solo le cifre numeriche decimali**. Gli ingressi da 1010 (0xA) a 1111 (0xF) producono delle **indifferenze**, perché non interessa il valore delle uscite.

Soluzione:

segmento a (con indifferenze)

$xy \setminus zw$	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	–	–	–	–
10	1	1	–	–

segmento g (con indifferenze)

$xy \setminus zw$	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	–	–	–	–
10	1	1	–	–

- **Segmento a** : implicanti primi $z, y'w', yw, x \Rightarrow a = z + y'w' + yw + x$ (**6 letterali**, contro i 14 senza indifferenze).
- **Segmento g** : implicanti primi $zw', x, yz', y'z, yw' \Rightarrow g = zw' + x + yz' + y'z$ (**7 letterali**, contro gli 11 senza indifferenze).

3.8.3 Come si usano le indifferenze

Inutile aggiungere implicanti nella copertura per coprire **solamente** indifferenze.

Si considerano le indifferenze come **1** quando si **cercano** gli implicanti.

Ma si considerano come **0** quando occorre **scegliere** quali implicanti utilizzare!!

Esempio.

$xy \backslash zw$	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	—	—
10	—	—	—	—

- Due implicanti primi: y e x . Prendendoli entrambi: $f = y + x$.
- Ma x copre solo indifferenze (gli 1 di xy sono già coperti da y): basta $f = y$.

3.8.4 Take away: indifferenze

- Le indifferenze sono combinazioni di ingresso per le quali **non interessa il valore dell'uscita** (per esempio perché la combinazione non si presenta mai): l'uscita può essere considerata alternativamente 0 oppure 1, a seconda della convenienza.
- **Si sfruttano** per semplificare l'espressione:
 - assumendo che l'indifferenza valga **1**, si possono **espandere gli implicanti**, che hanno quindi meno letterali;
 - durante la **scelta** degli implicanti si assume che valga **0**, così da non aggiungere implicanti inutili;
 - in particolare, **un implicante non sarà mai essenziale a causa di una indifferenza**.
- **Come ottenere le indifferenze?**
 - il caso più semplice è quando si sa che **certi ingressi non si presentano**;
 - il caso generale è molto più complesso e richiede l'**analisi di una rete** per identificare quando un'uscita è effettivamente indifferente.

3.9 Mappe a 5 e più variabili

3.9.1 Mappe a 5 variabili

- A 5 variabili (x, y, z, w, v) si ottengono **32 caselle**: anche usando il codice Gray è **impossibile mantenere le vicinanze geometriche sul piano**.
- Si usano **due tabelle da 4 variabili**, entrambe funzioni di x, y, z, w : la prima per $v = 0$, la seconda per $v = 1$ (**espansione di Shannon** sulla variabile v).

- Le vicinanze su v vanno considerate **tra una tabella e l'altra**: caselle che occupano la **stessa posizione** su entrambe le tabelle sono vicine e possono formare un termine; lo stesso vale per gli implicanti.
- Geometricamente: si immaginano le tabelle **poste una sopra all'altra**.

Esercizio: numeri primi (o quasi)

Ingresso: numero binario a **5 bit**. Uscita a 1 se il numero è **primo o divisibile per 3**, a 0 altrimenti.

Soluzione: mappa a 4 righe $\times$ 8 colonne ($a_4a_3 \backslash a_2a_1a_0$): in ogni casella è riportato il **numero decimale** corrispondente e le caselle in cui l'uscita vale **1** sono colorate.

$a_4a_3 \backslash a_2a_1a_0$	000	001	011	010	100	101	111	110
00	0	1	3	2	4	5	7	6
01	8	9	11	10	12	13	15	14
11	24	25	27	26	28	29	31	30
10	16	17	19	18	20	21	23	22

- **Adiacenze** tra colonne (da non confondere con la semplice vicinanza grafica):
 - **non sono vicine** le colonne 010 e 100 (al centro), né le colonne 000 e 110 (agli estremi);
 - **sono vicine** le colonne 000 e 010, le colonne 100 e 110 e, ovviamente, le colonne 000 e 100 (stessa posizione nelle due metà).
- **Espressione minima:** ci si deve tenere **tutti gli implicanti**:

$$p = a'_4a_0 + a'_3a_0 + a_2a_0 + a_1a_0 + a'_4a'_3a_1 + a'_3a'_2a_1 + a_4a_3a_2a_1 + a'_4a_3a_2a'_1 + a_4a_3a'_2a'_1a'_0$$

Osservazioni.

- Trovare l'espressione minima è un po' **tedioso** ed è facile sbagliare con tutti gli implicanti.
- Metodo alternativo: un'altra numerazione delle colonne, usando **direttamente il codice Gray**. Attenzione però: **le vicinanze cambiano** (per esempio le due colonne di mezzo sarebbero vicine) e non si può più pensare alle due mappe come sovrapposte.

3.9.2 Mappe a 6 e più variabili

- Si possono usare **4 mappe a 4 variabili**, ma in ogni caso non sono molto convenienti.
- Per più di 6 variabili **attenzione a come si ordinano le mappe**: conviene usare di nuovo il **codice Gray in modo gerarchico** anche per le variabili che indicizzano le mappe (esempio nelle slide: mappa abc/def , con la prima variabile di righe e colonne che divide la mappa in due metà e, dentro ciascuna, ordine Gray).
- Sono solo **più complesse geometricamente** e mettono a dura prova il colpo d'occhio; altrimenti non c'è nulla di nuovo.
- Alternativa: **spezzarle con Shannon!**

3.10 Altri modi di sintetizzare: NAND–NAND e NOR–NOR

3.10.1 Realizzazione NAND–NAND

- È conveniente usare **sempre lo stesso tipo di porta logica**: rende la realizzazione più omogenea in termini di **caratteristiche elettriche**.
- Le **porte invertenti** usano **meno transistori** di quelle non invertenti.
- Si ottiene la sintesi NAND–NAND partendo da quella AND–OR e applicando la **legge di De Morgan**:

$$\begin{aligned}f &= z + y'w' + yw + xz' \\f &= (z + y'w' + yw + xz')'' \\f &= ((z)'(y'w')'(yw)'(xz')')'\end{aligned}$$

- Gli **implicanti** sono **gli stessi** della sintesi AND–OR e si individuano allo stesso modo.
- **Procedimento pratico**:
 1. ad ogni **AND** si sostituisce una **NAND**;
 2. ad ogni **OR** si sostituisce una **NAND**;
 3. se un ingresso va **direttamente alla OR**, viene **negato**.

3.10.2 Realizzazione NOR–NOR

- Come prima, ma si parte dal **prodotto di somme**: gli implicanti si ottengono dagli **zeri** della funzione.
- Si applicano le leggi di De Morgan e di nuovo si sostituisce tutto con porte **NOR**, con l'avvertenza di **negare gli ingressi che vanno direttamente alla porta di uscita**.
- Esempio dalle figure delle slide: $(a + b)c = ((a + b)' + c')'$ (NOR sui primi due ingressi; il terzo, negato, va alla NOR di uscita).

3.10.3 Take away: alternative realizzative

- Vi sono **varie alternative realizzative**:
 - si possono differenziare per **tecnologia**;
 - a seconda della funzione possono produrre implementazioni **migliori o peggiori**;
 - talvolta è utile rappresentare tutto con **sole porte NAND**, per semplificare l'analisi da parte di strumenti automatici.
- **Procedimento standard**: in tutti i casi non cambia; è possibile passare da una rappresentazione a un'altra applicando semplici **proprietà dell'algebra booleana**.
- **Complessità**: sono possibili mappe con un gran numero di variabili, **difficili da manipolare a mano**.

4 Tempistiche e diagrammi temporali

4.1 Perché le tempistiche contano

- Una funzione può essere **giusta** e il circuito **non funzionare**: i circuiti hanno **ingressi che variano nel tempo** (qualcuno preme un pulsante, cambia la lettura di un sensore, i dati sono in forma seriale, ecc.).
- Anche le **uscite** cambiano nel tempo: passato il **transitorio** si stabilizzano sul valore dettato dalla funzione logica.
- Come influisce la rete sulle tempistiche delle uscite?
 - **ogni porta logica introduce un ritardo**;
 - la **somma dei ritardi** lungo il cammino del segnale definisce il ritardo finale dell'uscita.
- Interessa poter **rappresentare il comportamento del circuito durante la fase di transitorio**.

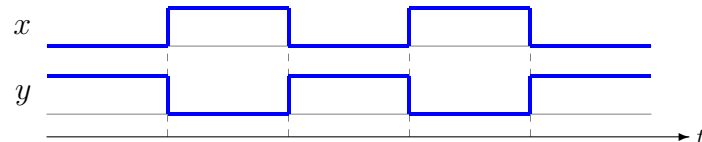
4.2 Diagramma temporale

- Rappresentazione dell'**evoluzione nel tempo dei valori dei nodi** della rete (come i diagrammi temporali in elettronica analogica).
- Normalmente non interessano i dettagli della fase di salita e discesa delle singole porte: si usano **solo due livelli**.
- Due versioni:
 1. **senza ritardi** delle porte: le variazioni degli ingressi si propagano *immediatamente* ai nodi interni e all'uscita $\Rightarrow$ utile per **verificare e debuggare la funzione logica**;
 2. **con i ritardi** delle porte: le variazioni degli ingressi impiegano del tempo per propagarsi ai nodi interni e all'uscita $\Rightarrow$ utile per **valutare le prestazioni**, trovare i **percorsi critici** e analizzare eventuali **anomalie**.

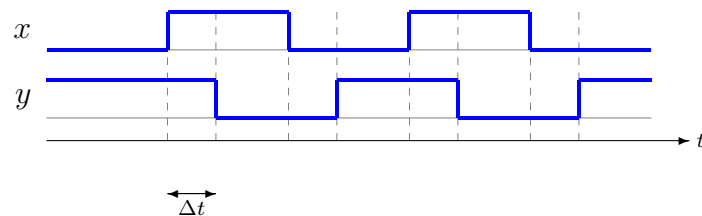
4.2.1 Diagramma dell'invertitore

- Due nodi: ingresso x e uscita y .
- **Senza ritardo** attraverso la porta: per ogni istante di tempo l'uscita ha valore **opposto** a quello dell'ingresso.
- **Con ritardo** Δt attraverso la porta: l'uscita raggiunge il valore finale solo **dopo il ritardo della porta**.

Invertitore senza ritardo



Invertitore con ritardo Δt



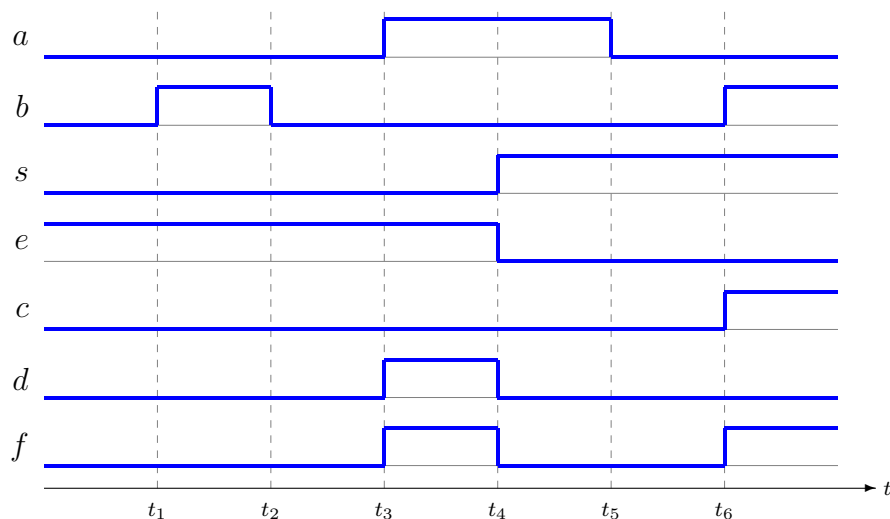
4.2.2 Rete logica: il multiplexer

Circuito della slide (multiplexer 2 a 1): $c = b \cdot s$ (AND), $e = s'$ (NOT), $d = a \cdot e$ (AND), $f = c + d$ (OR), cioè $f = bs + s'a$.

Passi per costruire il diagramma:

1. identificare i **nodì interni** (c , d , e);
2. definire gli **stimoli di ingresso** (a , b , s);
3. determinare i **valori dei restanti nodi**.

Multiplexer senza ritardi



Istante	Evento sugli ingressi	Effetto sulla rete
t_1	b sale	$s = 0$: c resta a 0, f resta a 0
t_2	b scende	nessun effetto
t_3	a sale	$e = 1 \Rightarrow d = 1 \Rightarrow f = 1$
t_4	s sale	$e = 0 \Rightarrow d = 0$; $c = 0$ perché $b = 0 \Rightarrow f = 0$
t_5	a scende	nessun effetto (d è già a 0)
t_6	b sale	$s = 1 \Rightarrow c = 1 \Rightarrow f = 1$

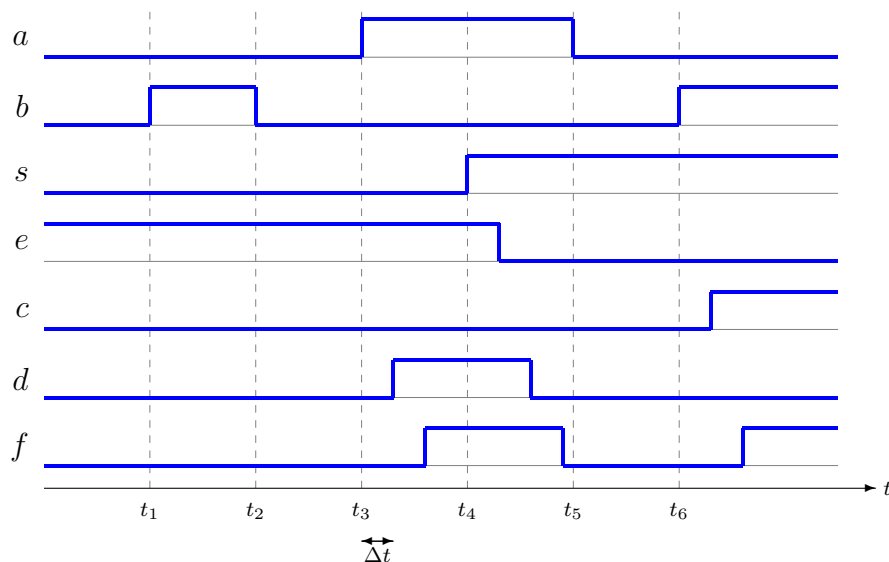
4.2.3 Come completare un diagramma

1. Se non succede niente, i **valori dei nodi non cambiano**; per convenzione gli ingressi sono **costanti per** $t < 0$; interessano solo i punti in cui *qualche* segnale cambia di valore.
2. Si valuta il valore di **tutti i nodi a** $t = 0$ applicando la funzione logica a partire dagli ingressi (si assume che a $t = 0$ tutto sia **a regime**).
3. Ci si sposta al **minimo tempo** t^* per il quale uno *qualunque* dei segnali (non solo gli ingressi) commuta di valore: si dice che il segnale ha un **evento** a t^* .
4. Si calcolano i **nuovi valori** nella rete e si fanno commutare i segnali dei nodi che cambiano valore, **aggiungendo nuovi eventi** al diagramma (tenendo eventualmente conto dei ritardi).
5. Ci si sposta al prossimo evento e si **ripete** la procedura fino ad aver esaurito tutti gli eventi.

4.2.4 Rete logica: il multiplexer con ritardi

- Si suppone, per semplicità, che le porte abbiano **tutte lo stesso ritardo** Δt .
- Si **trasla verso destra** il segnale all'uscita di ogni porta e si **accumulano i ritardi** della rete.

Multiplexer con ritardi



4.2.5 Osservazioni

- Il **ritardo** tra gli ingressi e l'uscita non è sempre lo stesso: dipende da *quale* ingresso cambia.
 - la variazione su a impiega $2\Delta t$ per arrivare all'uscita ($a \rightarrow d \rightarrow f$);
 - la transizione su s impiega $3\Delta t$ ($s \rightarrow e \rightarrow d \rightarrow f$).
- Per trovare il **massimo** e il **minimo** occorrerebbe provare tutte le possibili transizioni, ma esistono metodi più semplici che analizzano la **topologia** della rete.
- **Modelli di ritardo**: quello usato è molto semplice e lo si può complicare a piacere:
 - porte con **ritardi diversi** a seconda di quale ingresso cambia;
 - aggiustamento dei ritardi secondo il **carico** (fan-out);
 - tenuta in conto dei **tempi di salita e discesa**;
 - **modello inerziale**, che elimina brevi transizioni degli ingressi.

4.3 Alea (glitch)

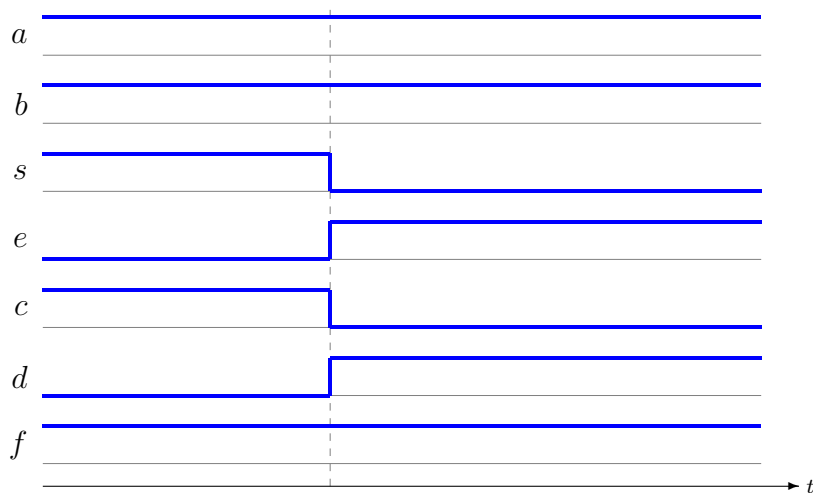
- Sono **brevi cambiamenti di valore** che si esauriscono con il transitorio.
- Sono dovute a **particolari combinazioni di ritardi** nelle reti logiche.
- Influiscono **solo sul transitorio**, ma possono dare molto fastidio nei circuiti **sensibili ai fronti** dei segnali (**circuiti sequenziali asincroni**).

4.3.1 Alea nel multiplexer

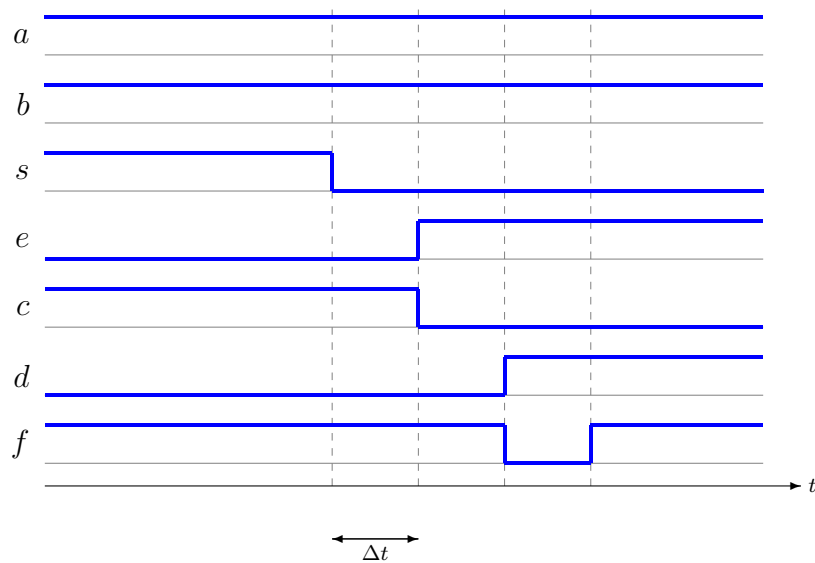
Caso considerato: $a = b = 1$ costanti, s passa da 1 a 0.

- **In assenza di ritardi**: un implicant si disattiva (c), l'altro si attiva (d) e l'uscita rimane al valore **1 costante**.
- **Con ritardi**: l'implicant c si **disattiva prima** che l'implicant d possa mantenere l'uscita a livello alto $\Rightarrow$ l'uscita si porta **per un periodo a livello basso** (alea).

Alea nel multiplexer: senza ritardi



Alea nel multiplexer: con ritardi



4.3.2 Alee sulla mappa di Karnaugh

- Si verifica una alea **solo se** gli ingressi cambiano in modo da **passare da un implicante all'altro**.
- Inoltre i ritardi devono essere tali da **disattivare il primo prima di attivare il secondo**.
- Nella mappa di Karnaugh si **salta da una casella a un'altra senza stare in uno stesso implicante**.
- Se invece **non si cambia implicante** non si può verificare una alea.

$s \backslash ab$	00	01	11	10
0	0	0	1	1
1	0	1	1	0

verde: implicante $s'a$ **giallo:** implicante sb salto da $(s, a, b) = (1, 1, 1)$ a $(0, 1, 1)$,
cioè da sb a $s'a$

4.3.3 Come prevenire le alee

- È sufficiente aggiungere **implicanti di collegamento** (nell'esempio, l'implicante che copre le due caselle $ab = 11$).
- La realizzazione **non è più minima**.
- **Indispensabile** per i circuiti **asincroni**.
- Utile nei circuiti **sincroni** solo per ridurre eventualmente i **consumi**.

$s \backslash ab$	00	01	11	10
0	0	0	1	1
1	0	1	1	0

arancio: caselle $ab = 11$, coperte *anche* dall'implicante di collegamento ab

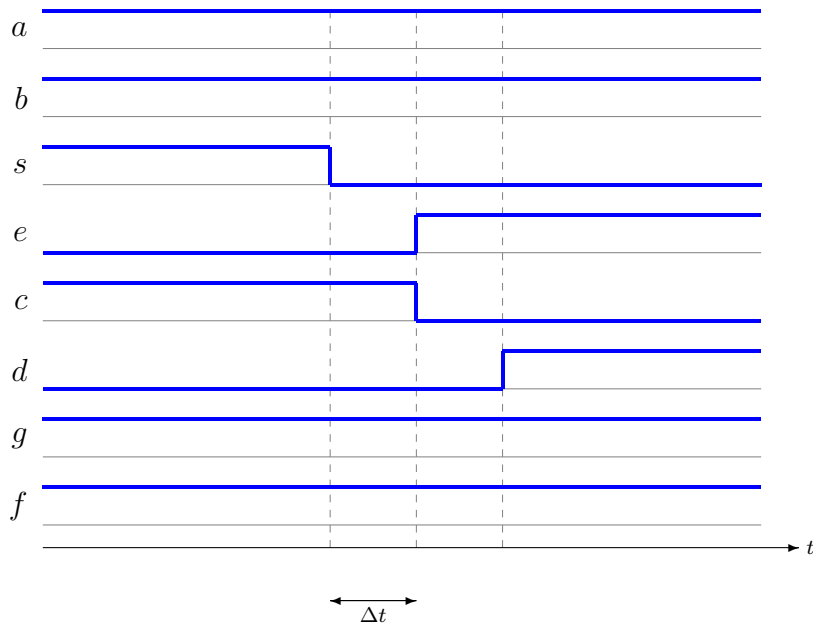
Esercizio: diagramma temporale del multiplexer senza alea

Costruire il diagramma temporale del multiplexer a cui è stato aggiunto l'implicante di collegamento, nel caso $a = b = 1$ e s che passa da 1 a 0 (con ritardi uguali per tutte le porte).

Soluzione:

- L'implicante di collegamento è $g = ab$ e la funzione diventa $f = bs + s'a + ab$.
- Con $a = b = 1$ il nodo g vale **1 per tutto il tempo**: quando c si disattiva e d non si è ancora attivato, è g a **mantenere f a livello alto**.
- Di conseguenza f resta a 1 e l'**alea scompare**.

Multiplexer con implicante di collegamento: nessuna alea



4.4 Take away

- Le tempistiche influenzano il funzionamento del circuito
 - i ritardi introducono **asimmetrie** nella rete logica;
 - diversi modelli, con differente accuratezza, aiutano a stimare le tempistiche;
 - utile conoscere i **ritardi minimi e massimi** attraverso una rete logica.
- Le asimmetrie danno luogo a fenomeni di alee

- le alee, o **glitch**, possono **aumentare i consumi di potenza**;
- sono particolarmente insidiose in certi tipi di circuiti **sensibili ai fronti** dei segnali;
- si possono eliminare introducendo elementi che **neutralizzano le transizioni spurie**.